Chapter 2

# BRUTE FORCE APPROACHES

*) Brute force method is one of the most widely used algorithmic design technique. This is the simplest method among all methods.

*) Brute force method is a straight forward approach to solve given problem. Usually it is based on problem statement & definitions of concepts involved.

*) The concept or the basic idea of this technique is "just do it".

*) Some of the well known problems which can be solved using this method are

1) Consecutive integer checking method for Computing GCD
2) Definition based algorithm for matrix multiplication
3) Element uniqueness problem
4) Linear search (also called as sequential search)
5) Selection sort
6) Bubble sort
7) Brute force string matching
8) Travelling salesman problem
9) Knapsack problem, etc.

MADHUSUDHAN M.V.
Dept. of CSE

*) Advantages of brute force method are

1) applicable to very wide range of problems such as finding sum of n numbers, computing power, GCD, etc
2) using this method, easy & simplest algorithms can be written. Ex: selection sort, bubble sort, matrix multiplication, etc
3) Cost effective
4) very useful, especially for solving small size problems.

*) Disadvantages of brute force method are

1) Rarely yields efficient algorithms
2) most of the brute force algorithms are slower
3) this method is in-efficient in general.

## 2.1 Selection Sort

The process of re-arranging the elements in an ascending or descending order is called sorting.

### procedure of selection sort

*) find the smallest element in the given list of 'n' elements & exchange it with the 1st element.

*) find the 2nd smallest element in the remaining n-1 elements & exchange it with the 2nd element & so on.

Example

a

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 89 | 45 | 29 | 168 | 80 |

n=5

MADHUSUDHAN M.V.
Dept. of CSE

| | 0 | 1 | 2 | 3 | 4 | |
|---|---|---|---|---|---|---|
| 1st pass | 29 | 45 | 89 | 168 | 80 | 29 is the smallest & exchange it with 1st element |
| 2nd pass | 29 | 45 | 89 | 168 | 80 | 45 is the 2nd smallest & exchange it with 2nd element. But here, 2nd smallest & 2nd element both are 45 only. |
| 3rd pass | 29 | 45 | 80 | 168 | 89 | 80 is 3rd smallest & exchange it with 3rd element (i.e., 89 with 80). |
| 4th pass | 29 | 45 | 80 | 89 | 168 | 89 is 4th smallest & exchange it with 4th element (i.e., 89 with 168). |

For 5 elements, we need 4 passes. Similarly, for 6 elements, we need 5 passes. In general, for 'n' elements we need n-1 passes.

After 1st pass, 1st element get sorted. After 2nd pass, 1st 2 elements get sorted. After 3rd pass, 1st 3 elements get sorted. So, in general, after $i^{th}$ pass, 1st 'i' elements get sorted. So, to sort 'n' elements we need n-1 passes.

ALGORITHM :- Selection_Sort (a[0.....n-1])

// purpose :- To sort the given array using selection sort

// input :- An array a[0....n-1] to be sort

// output :- Array a[0...n-1] whose elements sorted in ascending order

```
for i ← 0 to n-2 do
    min ← i
    for j ← i+1 to n-1 do
        if a[j] < a[min] then
            min ← j
        end if
    end for
    Swap a[i] & a[min]
end for
```

MADHUSUDHAN M.V.
Dept of

Analysis

*) problem size → number of elements to sort, 'n'

*) Basic operation → Comparison operation, a[j] < a[min]

*) In whatever the order we can give the array elements, we need to check & sort whole array.

That means, number of times the basic operation is executing depends only on the problem size. So, we need to find only $\Theta$.

*) The number of times the basic operation is executing is

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

$$= \sum_{i=0}^{n-2} n-1-(i+1)+1 = \sum_{i=0}^{n-2} n-i-1$$

putting $i = 0, 1, 2, 3, \ldots n-2$, we get

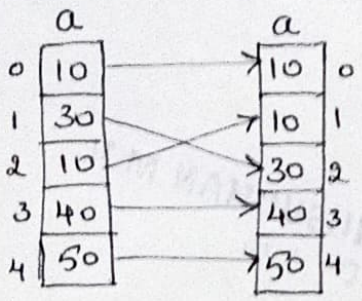
$$C(n) = (n-1) + (n-2) + (n-3) + \ldots + 3 + 2 + 1$$

$$= \frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2}$$ . Here, $n^2$ is dominating term

$\therefore$ $C(n) \in \Theta(n^2)$

*) Efficiency of selection sort in terms of basic operation (assignment) is $\Theta(n^2)$. Whereas, efficiency of selection sort in terms of swapping is $\Theta(n)$ because it swaps only once for each iteration of outermost for loop.

*) If any sorting algorithm preserves the relative order of 2 equal elements, then that sorting algorithm is said to be stable.




Selection sort algorithm is an stable algorithm because it preserves the relative order of any 2 equal elements.

*) If any sorting algorithm requires very few extra memory other than the given array & n, then that sorting algorithm is ~~called~~ said to be in place.

Selection sort algorithm is in place as it requires very few extra memory (only for i, j & swap).

*) Disadvantages of selection sort.

1) Its time complexity is in the order of $\Theta(n^2)$ & hence, it is not an efficient sorting algorithm especially, when there is too many elements to sort.

2) Eventhough the given array is already sorted, then also it takes n-1 passes to sort.

MADHUSUDHAN M.V.
Dept. of CSE

## 2.2 Bubble Sort.

*) This is the simplest & more straightforward method of sorting.

*) In this method, adjacent elements of the list are compared.

*) If they are out of order (if 1st element is greater than 2nd) then we will exchange them.

*) Comparing like this repeatedly till the $n^{th}$ element, we will find the largest number & which will be in the last position.

*) At the end of pass1, the largest element will be in last position.

*) After 2nd pass, 2nd largest element will be in 2nd last position of the list. & so on.

*) After n-1 number of passes, the list will be sorted.

*) So, in general, $i^{th}$ pass of bubble sort can be represented by the following sequence

MADHUSUDHAN M.V.
Dept. of CSE

$A_0, A_1, A_2 \cdots A_j \overset{?}{\leftrightarrow} A_{j+1} \cdots A_{n-i-1} \mid A_{n-i}, \cdots A_{n-2}, A_{n-1}$

Unsorted elements | Sorted elements (largest elements in their final position).

Example: a

| 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 89 | 45 | 29 | 168 | 80 |

n=5

89 45 29 168 80 → 89 & 45 is out of order, so exchange them

45 89 29 168 80 → 89 & 29 is out of order, so exchange them

45 29 89 168 80 → 89 & 168 is in order, so no need to exchange

45 29 89 168 80 → 168 & 80 is out of order, so exchange them

45 29 89 80 (168) → After pass1, largest element is in its final position

pass1

29 45 89 80 → 45 & 29 are out of order, so exchange them

29 45 89 80 → 45 & 89 are in order, so need to exchange

29 45 89 80 → 89 & 80 are out of order, so exchange them

29 45 80 (89) → At the end of pass2, 2nd largest element is in its final position

pass2

29 45 80 → 29 & 45 are in order, so no need to exchange.

29 45 80 → 45 & 80 are in order, so no need to exchange.

29 45 (80) → After pass3, 3rd largest element is in its final position

pass3

29 45 → 29 & 45 are in order, so no need to exchange

29 (45) → After pass4, 4th largest element is in its final position

pass4

29 45 80 89 168 → After pass4, all 5 elements get sorted.

So, for n=5, we need 4 passes to sort. Similarly,
for n=6, we need 5 passes to sort.

⋮

In general, for 'n' number of elements, we need n-1 passes.

ALGORITHM:- Bubble_Sort (a[0.....n-1])
//purpose:- To sort the given list of elements using bubble sort method
//Input:- An array a[0....n-1] to be sorted
//Output:- Sorted array a[0.....n-1]

```
for i ← 0 to n-2 do
    for j ← 0 to n-2-i do
        if a[j+1] < a[j]
            Swap a[j+1] & a[j]
        end if
    end for
end for
```

MADHUSUDHAN M.V.
Dept. of CSE

Analysis

*) problem size ⟶ 'n'; number of elements to be sorted
*) Basic operation ⟶ Comparison operation, a[j+1] < a[j]
*) Since, the number of times the basic operation is executing depends only on the problem size, we need to find only "Θ".
*) The number of times the basic operation is executing is given by

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1$$

$$= \sum_{i=0}^{n-2} n-2-i-0+1 = \sum_{i=0}^{n-2} n-1-i$$

Substituting, i = 0, 1, 2, ..... n-2, we get

$$C(n) = (n-1)+(n-2)+(n-3)+ \cdots +3+2+1$$

$$= \frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2}.$$ Here, $n^2$ is more dominating

$$\therefore C(n) \in \Theta(n^2)$$

*) The time complexity of bubble sort algorithm is $\theta(n^2)$.

*) In this algorithm, swapping operation is also executing more number of times.

*) The number of times, the swaping operation is executing depends on how we give the input elements.

*) Suppose if we give the array elements (input elements) in decreasing order, then at every time of comparison, swaping operation is get executed. That means, if we give the input elements in decreasing order, then the number of times the swaping operation is executing is same as number of comparison operation. So, this is at the worst-case.

$\therefore S_{worst}(n) = C(n) \in \theta(n^2)$

*) Suppose, if we give the input elements in ascending order itself, then swaping operation is not going to execute even once. This is at the best case.

*) Drawback of this bubble sort algorithm

MADHUSUDHAN M.V.
Dept. of CSE

It is not an efficient sorting algorithm because this algorithm runs for $n^2$ order even if we give the input elements in the ascending order.

Improved version of the bubble sort algorithm.

Suppose, if we give the input elements in an ascending order, then swapping operation is not going to be executed even once in the 1st pass. This is the indication, that, the input elements are already sorted. In this case, we should stop the algorithm immediately after the 1st pass. In this way, we can improve the efficiency.

ALGORITHM:- Improved_version_bubble_sort (a[0.....n-1])

//purpose:- To sort the array a[0...n-1] efficiently using bubble sort

//Input:- An array a[0.....n-1] which is to be sorted

//Output:- Sorted array a[0.....n-1]

```
for i←0 to n-2 do
    flag←0
    for j←0 to n-2-i do
        if a[j+1] < a[j] then
            swap a[j+1] & a[j]
            flag←1
        end if
    end for
    if flag=0 then
        break
    end if
end for
```

MADHUSUDHAN M.V.
Dept. of CSE

Analysis

*) problem size → 'n', number of elements to be sorted

*) Basic operation → Comparison Operation "a[j+1] < a[j]"

*) Since, the number of times the basic operation is executing depends on how we give the input elements, we need to find O & Ω separately.

*) Worst-case

If we give the input elements in decreasing order, then it results in worst-case possibility.

In the worst-case (O) & average case (Θ), efficiency of this improved version remains same as older (previous) version.

∴ $C_{worst}(n) = C_{avg}(n) = n^2$ (OR) $C_{avg}(n) \in \Theta(n^2)$ & $C_{worst}(n) \in O(n^2)$

*) Best-case

If the input elements are already in ascending order, then it is the best-case possibility.

If no swapping operation is done in the 1st pass, then it indicates elements are already sorted. So, we need atleast one pass to check whether elements are sorted or not.

∴ for j ← 0 to n-2-i do

if a[j+1] < a[j] then

$$C_{best}(n) = \sum_{j=0}^{n-2-i} 1 = n-2-i-0+1 = n-1-i \longrightarrow (1)$$

If items are already sorted, then outer for loop will run only once for i=0. Therefore equation (1) becomes

$C_{best}(n) = n-1-0 = n-1$

∴ $C_{best}(n) \in \Omega(n)$.

MADHUSUDHAN B.V.
Dept. of CSE

*) Before Sorting / After Sorting

| | Before Sorting | After Sorting |
|---|---|---|
| a[0] | 10 | 10 |
| a[1] | 30 | 10 |
| a[2] | 10 | 30 |
| a[3] | 40 | 40 |
| a[4] | 50 | 50 |

Bubble sort algorithm is stable, because it preserves the relative order of 2 equal elements.

Before & after sorting, the position of 1st 10 is less than the position of 2nd 10 i.e., the relative order of 2 equal elements has not been changed.

*) Bubble sort algorithm is in-place because it uses few extra memory for i, j & temp apart from array 'a' & n.

Disadvantages

*) It runs slowly & hence it is not efficient

*) Even if the elements are already sorted, then also it requires n-1 number of passes to sort (for older version).

## 2.3 Sequential Search (or) Linear Search

This is the most simplest searching algorithm which uses brute force technique.

problem statement

Given the set of elements & an key element. We need to search this key element in the given list of elements. If we found the key element, ~~stop~~ return the position of element in the list & stop searching; otherwise return -1.

This algorithm simply compares the successive elements of a given list with given key element until either a match is encountered (successfull) & it reaches the full length of list without finding the match (unsuccessfull search).

In order to find the worst-case efficiency, we will add (append) the key element at the end of list, so that it will search the whole list.

ALGORITHM: Linear_Search (a[0....n-1], K)

//purpose: To search the key element 'K' in the given array a[0...n-1]

//Input: Array of 'n' elements & key element 'K'.

//Output: position of element in a[0...n-1] whose value is equal to 'K' or -1 if no such element is found.

```
a[n] ← K
i ← 0
while a[i] ≠ K do
        i ← i+1
end while
if i < n
    return i
else
    return -1
```

MADHUSUDHAN M.V.
Dept. of CSE

## Analysis

*) problem size ⟶ number of elements in the list 'n'.

*) Basic operation ⟶ Comparison operation "a[i] ≠ K".

*) The number of times the basic operation is executing depends not only on the problem size but also depends on how we give the input elements. So, we need to find O & Ω.

### Worst-case

Worst-case happens in 2 situations

1) if key element is found at the last position of the array
2) if key element is not found.

In both the cases, algorithm makes as ~~many~~ many number of comparisons as number of elements in the array.

$$C_{worst}(n) = \sum_{i=0}^{n-1} 1 = n-1-0+1 = n$$

$$\therefore C_{worst}(n) \in O(n)$$

MADHUSUDHAN M.V.
Dept. of CSE

### Best-case

Best-case happens if key element is found at the 1st position of the array itself. In this case, it comparison operation will be executed only once.

$$\therefore C_{best}(n) \in \Omega(1)$$

### Advantages

*) Simple & straight forward method

*) Very useful especially the number of elements are less & if the elements are not in order.

### Disadvantages

*) Not an efficient algorithm

*) it will take more time when there is more number of elements are there in list.

## 2.4 Brute-force string matching (pattern matching)

**problem statement :-**

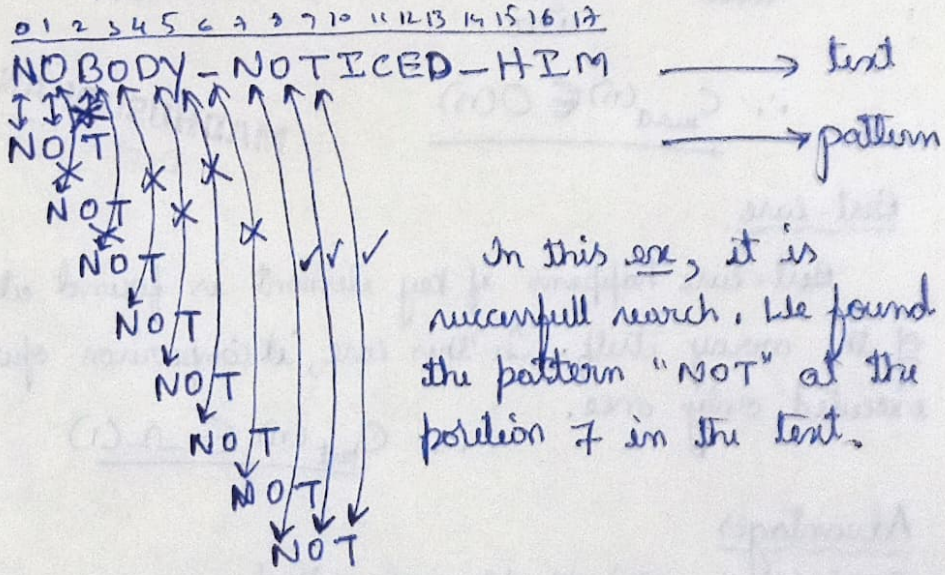
Given a string of characters called "text" of size 'n' & an array of 'm' characters ($m \leq n$) called "pattern". We need to search the given pattern in the given text.

**Solution :-** 'm' characters of the pattern is matched one by one with 'n' characters of the text from left to right. If mismatch occurs, then pattern is shifted towards right by one position & again characters of text & pattern are matched one by one.

If match is found, then index (position) of 1st match of the text is returned as a result.

If no matching is found after checking the entire text, then -1 is returned.

**Example :-**




In this ex, it is succesfull search. We found the pattern "NOT" at the position 7 in the text.

MADHUSUDHAN M.V.
Dept. of CSE

To start, align the pattern against the text from starting character of the text. Now, Compare the adjacent characters of text & pattern. If both matches then Continue Comparing the next Corresponding characters of text & pattern untill we found all 'm' characters of pattern.

At any stage, if mismatch occurs, simply shift the pattern towards its right by one character & again compare the corresponding characters from beginning of pattern.

If we not found the pattern in the text, then we have to shift the pattern "n-m" number of times.

for ex, assume we need to search the pattern "WAS" in the given text, but it is not present.

After shifting the pattern "WAS" 15 times, pattern is aligned against the text as shown below.

```
NOBODY_NOTICED_HIM
      &
      WAS
```

Again, at the 1st comparison itself, it fails. After shifting the pattern by one position towards its right, then it is as shown below

```
NOBODY_NOTICED_HIM
      &
       WAS
```

MADHUSUDHAN M.V.
Dept. of CSE

But, here, there is no enough characters in the text. So, no need to shift the pattern after "n-m" number of shifts.

Algorithm: Brute_force_String_matching (T[0...n-1], p[0...m-1])
//purpose: To find the pattern of m characters in the text of n characters
//Input: An array T[0...n-1] of n characters representing "text"
An array p[0...m-1] of m characters representing "pattern"
Always m≤n
//output: position of the 1st character in the text that starts the 1st matching substring if search is successful; otherwise return -1.

```
for i ← 0 to n-m do
    j ← 0
    while j<m and p[j]=T[i+j] do
        j ← j+1
    end while
    if j=m then
        return i
    end if
end for
return -1
```

MADHUSUDHAN M.V.
Dept. of CSE

## Analysis

*) problem size ⟶ Size of text 'n' & size of the pattern 'm'

*) Basic operation ⟶ Comparison operation "p[j]=T[i+j]"

*) Since, the number of times the basic operation is executing depends on how we give the ~~elements~~ inputs, text & pattern, we need to find O & Ω seperately.

## Worst-case analysis

Worst-case situation occurs ~~is~~

1) if the pattern is present at the end of the text
2) if the pattern is not present in the text.

```
for i ← 0 to n-m do
    j ← 0
    while j<m and p[j]=T[i+j] Can be
```

written as

```
for i ← 0 to n-m do
    for j ← 0 to m-1 do
        if p[j]=T[i+j]
```

$$C_{worst}(n) = \sum_{i=0}^{n-m} \sum_{j=0}^{m-1} 1$$

$$C_{worst}(n) = \sum_{i=0}^{n-m} m \text{ (for } 0 \text{ to } n-m) = \sum_{i=0}^{n-m} m = m \sum_{i=0}^{n-m} 1$$

$$= m(n-m-0+1) = mn - m^2 + m$$

Since, $n > m$, the term "mn" dominates more than other terms.

$\therefore C_{worst}(n) \in O(mn)$

MADHUSUDHAN M.V.
Dept. of CSE

Best-case analysis ($\Omega$)

Best-case occurs if the pattern is found at the beginning of the text itself. At this time, it compares only the size of the pattern i.e., $m$ times.

$\therefore C_{best}(n) \in \Omega(m)$

Average case ($\Theta$)

In the average case, this algorithm makes comparisons more than number of times than in the best case ($> m$) & less number of times than in the worst-case ($< mn$).

Usually, average case is obtained after few comparisons by few shifting.

At the worst case, algorithm makes $n-m$ number of shifts. For average case, it is lesser than $n-m$. In each shift, we have to check for $\leq m$ times. So, average case efficiency is given by $C_{avg}(n) = \Theta(n+m) \approx \Theta(n)$

$\therefore C_{avg}(n) \in \Theta(n)$

Since, this algorithm makes more number of shifts & comparisons operations, this is not an efficient algorithm for pattern matching.

Boyer-Moore algorithm & Karpel algorithm are the efficient algorithms for string matching problems, which we will discuss in the later chapters.

MADHUSUDHAN M.V.
Dept. of CSE